Laboratory work 12

Styling the Frontend with MUI

This chapter explains how to use **Material U**I (**MUI**) components in our frontend. We will use the `Button` component to show styled buttons. We will also use MUI icons and the `IconButton` component. The input fields in our modal forms will be replaced by `TextField` components.

In this chapter, we will cover the following topics:

- Using the MUI `Button` component
- Using the MUI `Icon` and `IconButton` components
- Using the MUI `TextField` component

At the end of the chapter, we will have a professional and polished user interface with minimal code changes in our React frontend.

Technical requirements

The Spring Boot application that we created in *Chapter 5*, *Securing Your Backend*, is required, together with the modification from *Chapter 12*, *Setting Up the Frontend for Our Spring Boot RESTful Web Service* (the unsecured backend).

We also need the React app that we used in *Chapter 13*, *Adding CRUD Functionalities*.

The code samples available at the following GitHub link will also be required: `https://github. com/PacktPublishing/Full-Stack-Development-with-Spring-Boot-3and-React-FourthEdition/tree/main/Chapter14`.

Using the MUI Button component

Our frontend already uses some Material UI components, such as `AppBar` and `Dialog`, but we are still using a lot of HTML elements without any styling. First, we will replace HTML button elements with the Material UI `Button` component.

Execute the following steps to implement the `Button` component in our **New car** and **Edit car** modal forms:

1. Import the MUI `Button` component into the `AddCar.tsx` and `EditCar.tsx` files:

```
// AddCar.tsx & EditCar.tsx
```

```
import Button from '@mui/material/Button';
```

2. Change the buttons to use the `Button` component in the `AddCar` component. We are using '`text`' buttons, which is the default `Button` type.

If you want to use some other button type, such as '`outlined`', you can change it by using the `variant` prop (`https://mui.com/material-ui/api/button/#Button-prop-variant`).

The following code shows the `AddCar` component's `return` statements with the changes:

```
// AddCar.tsx return(
  <>
    <Button onClick={handleClickOpen}>New Car</Button>
    <Dialog open={open} onClose={handleClose}>
      <DialogTitle>New car</DialogTitle>
      <CarDialogContent car={car} handleChange={handleChange}/>
      <DialogActions>
         <Button onClick={handleClose}>Cancel</Button>
         <Button onClick={handleSave}>Save</Button>
</DialogActions>
    </Dialog>
  </>
);
```

3. Change the buttons in the `EditCar` component to the `Button` component. We will set the **Edit** button's `size` to `"small"` because the button is shown within the car grid. The following code shows the `EditCar` component's `return` statements with the changes:

```
// EditCar.tsx return(
  <>
    <Button size="small" onClick={handleClickOpen}>
      Edit
    </Button>
    <Dialog open={open} onClose={handleClose}>
      <DialogTitle>Edit car</DialogTitle>
      <CarDialogContent car={car} handleChange={handleChange}/>
      <DialogActions>
        <Button onClick={handleClose}>Cancel</Button>
        <Button onClick={handleSave}>Save</Button>
      </DialogActions>
    </Dialog>
  </>
);
```

4. Now, the car list looks like the following screenshot:

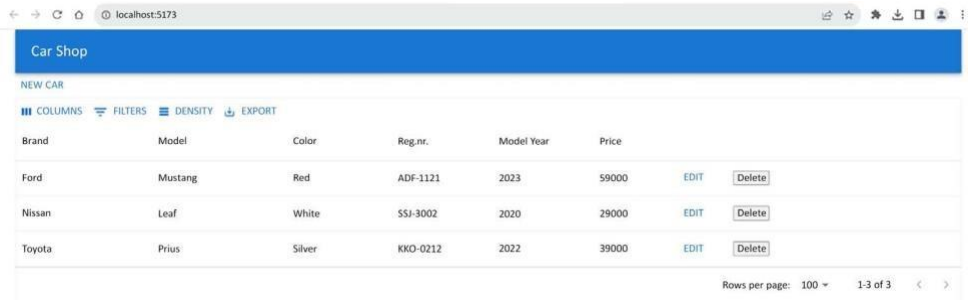

Figure 14.1: The Carlist buttons

The modal form buttons should look like the following:

Figure 14.2: The form buttons

Now, the buttons in the add and edit form have been implemented using the MUI `Button` component.

Using the MUI Icon and IconButton components

In this section, we will use the `IconButton` component for the **EDIT** and **DELETE** buttons in the grid. MUI provides pre-built SVG icons that we have to install by using the following command in the terminal:

```
npm install @mui/icons-material
```

Let's first implement the **DELETE** button in the grid. The MUI `IconButton` component can be used to render icon buttons. The `@mui/icons-material` package, which we just installed, contains lots of icons that can be used with MUI.

You can find a list of icons available in the MUI documentation (`https://mui.com/materialui/ material-icons/`). There is a search functionality, and if you click any of the icons in the list, you can find the correct import statement for a specific icon:

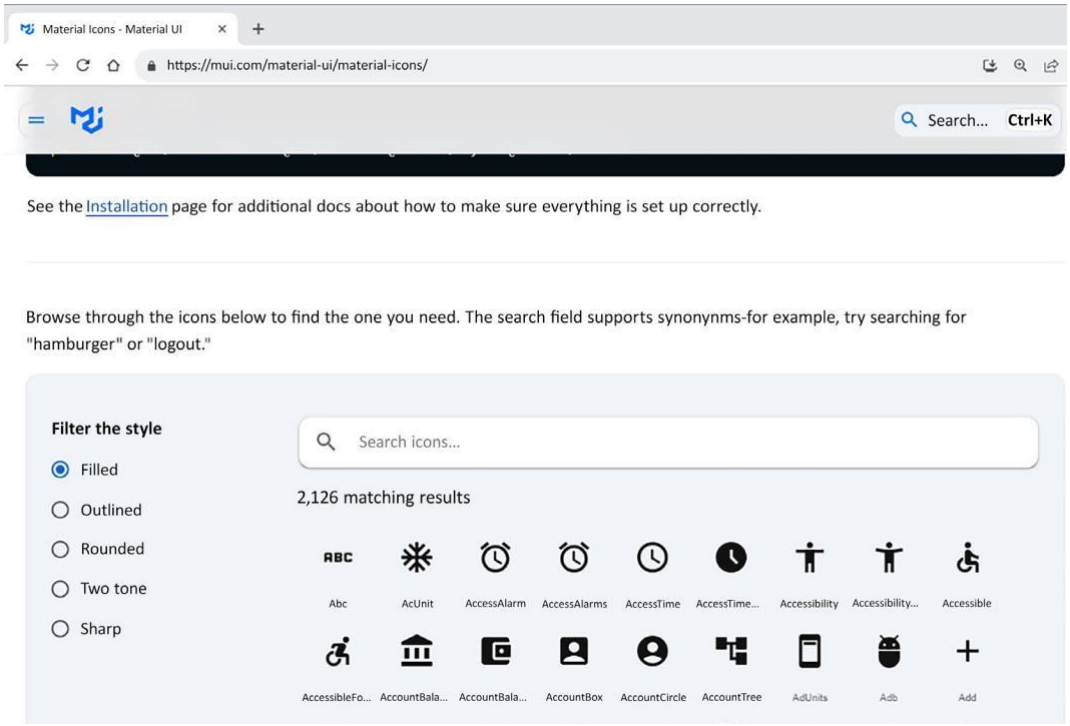

Figure 14.3: Material Icons

We need an icon for our **DELETE** button, so we will use an icon called `DeleteIcon`:

1. Open the `Carlist.tsx` file and add the following imports:

```
// Carlist.tsx import IconButton from
'@mui/material/IconButton'; import DeleteIcon from
'@mui/icons-material/Delete';
```

2. Next, we will render the `IconButton` component in our grid. We will modify the **DELETE** button in the code where we define the grid columns. Change the `button` element to the

   `IconButton` component and render the `DeleteIcon` inside the `IconButton` component. Set both the button and icon size to small. The icon buttons don't have an accessible name, so we will use `aria-label` to define a string that labels our delete icon button. The `arialabel` attribute is only visible to assistive technologies such as screen readers:

```
// Carlist.tsx const columns:
GridColDef[] = [
  {field: 'brand', headerName: 'Brand', width: 200},
  {field: 'model', headerName: 'Model', width: 200},
  {field: 'color', headerName: 'Color', width: 200},
  {field: 'registrationNumber', headerName: 'Reg.nr.', width: 150},
  {field: 'modelYear', headerName: 'Model Year', width: 150},
  {field: 'price', headerName: 'Price', width: 150},
  {     field: 'edit',     headerName: '',
width: 90,     sortable: false,     filterable:
false,     disableColumnMenu: true,
renderCell: (params: GridCellParams) =>
<CarForm mode="Edit" cardata={params.row} />
  },   {     field: 'delete',     headerName: '',     width:
90,     sortable: false,     filterable: false,
disableColumnMenu: true,     renderCell: (params:
GridCellParams) => (       <IconButton aria-label="delete"
size="small"         onClick={() => {           if
(window.confirm(`Are you sure you want to delete
${params.row.brand} ${params.row.model}?`))
mutate(params.row._links.car.href)
        }}>
        <DeleteIcon fontSize="small" />
      </IconButton>
    ),
  },
];
```

3. Now, the **DELETE** button in the grid should look like the following screenshot:

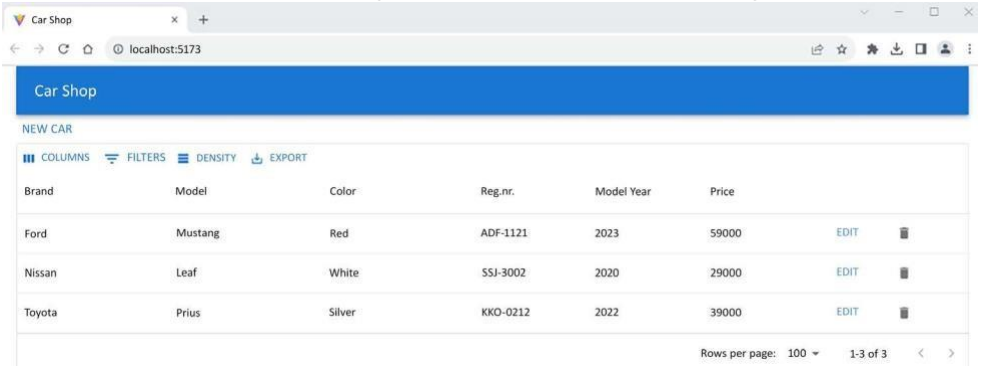

Figure 14.4: The Delete icon button

4. Next, we will implement the **EDIT** button using the `IconButton` component. Open the `EditCar.tsx` file and import the `IconButton` component and the `EditIcon` icon:

```
// EditCar.tsx import IconButton from
'@mui/material/IconButton'; import EditIcon from
'@mui/icons-material/Edit';
```

5. Then, render the `IconButton` and `EditIcon` in the `return` statement. The button and icon size are set to small, as with the delete buttons:

```
// EditCar.tsx
return(
  <>
    <IconButton aria-label="edit" size="small"
onClick={handleClickOpen}>       <EditIcon
fontSize= "small" />
    </IconButton>
    <Dialog open={open} onClose={handleClose}>
      <DialogTitle>Edit car</DialogTitle>
      <CarDialogContent car={car} handleChange={handleChange}/>
      <DialogActions>
        <Button onClick={handleClose}>Cancel</Button>
        <Button onClick={handleSave}>Save</Button>
      </DialogActions>
    </Dialog>
  </>
);
```

6. Finally, you will see both buttons are rendered as icons, as shown in the following screenshot:

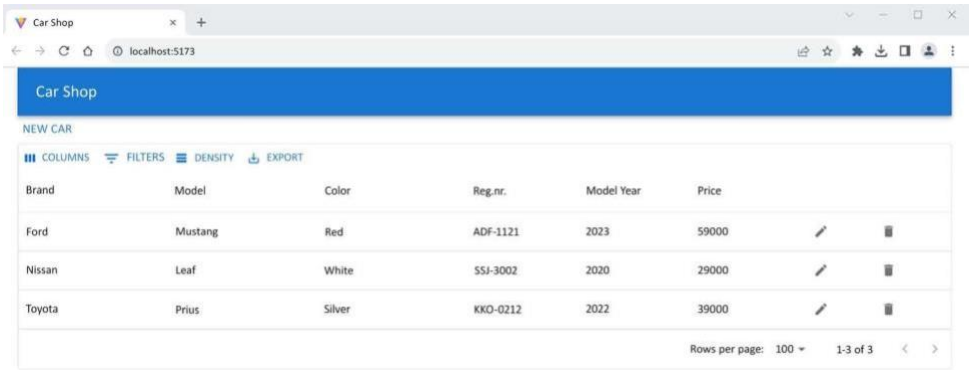

Figure 14.5: Icon buttons

We can also add **tooltips** to our edit and delete icon buttons using the `Tooltip` component. The `Tooltip` component wraps the component to which you want to attach the tooltip. The following example shows how to add a tooltip to the edit button:

1. First, import the `Tooltip` component by adding the following import to your `EditCar` component:

```
import Tooltip from '@mui/material/Tooltip';
```

2. Then, use the `Tooltip` component to wrap the `IconButton` component. The `title` prop is used to define the text that is shown in the tooltip:

```
// EditCar.tsx

<Tooltip title="Edit car">
  <IconButton aria-label=
onClick={handleClickOpen}>
  </IconButton>
</Tooltip>
```

```
edit" size="small"
    <EditIcon
fontSize= "small" />
```

3. Now, if you hover your mouse over the edit button, you will see a tooltip, as shown in the following screenshot:

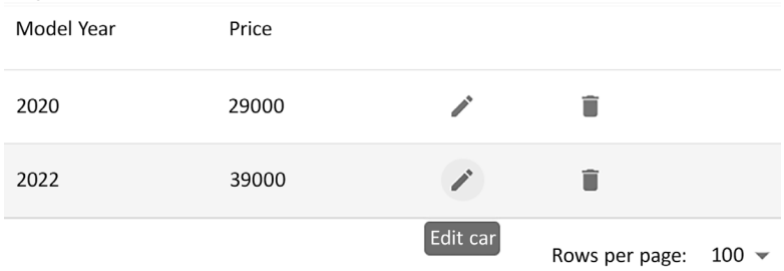

Figure 14.6: Tooltip

Next, we will implement text fields using the MUI `TextField` component.

Using the MUI TextField component

In this section, we'll change the text input fields in the modal forms to the MUI `TextField` and `Stack` components:

1. Add the following import statements to the `CarDialogContent.tsx` file. `Stack` is a onedimensional MUI layout component that we can use to set spaces between text fields:

```
import TextField from '@mui/material/TextField'; import
Stack from '@mui/material/Stack';
```

2. Then, change the input elements to the `TextField` components in the add and edit forms. We are using the `label` prop to set the labels of the `TextField` components. There are three different variants (visual styles) of text input available, and we are using the `outlined` one, which is the default variant. The other variants are `standard` and `filled`. You can use the `variant` prop to change the value. The text fields are wrapped inside the `Stack` component to get some spacing between the components and to set the top margin:

```
// CarDialogContent.tsx return
(
  <DialogContent>
      <Stack    spacing={2}
      mt={1}>
        <TextField
        label="Brand"
        name="brand"
    value={car.brand}
    onChange={handleChange}/>
        <TextField
        label="Model"
        name="model"
    value={car.model}
    onChange={handleChange}/>
      <TextField label="Color" name="color"
value={car.color} onChange={handleChange}/>      <TextField
label="Year" name="modelYear"        value={car.modelYear}
onChange={handleChange}/>      <TextField label="Reg.nr."
name="registrationNumber"        value={car.registrationNumber}
onChange={handleChange}/>
      <TextField label="Price" name="price"
value={car.price} onChange={handleChange}/>
    </Stack>
  </DialogContent>
);
```

You can read more about spacing and the units that are used at `https://mui.com/system/spacing/`.

3. After the modifications, both the add and edit modal forms should look like the following because we are using the `CarDialogContent` component in both forms:

Figure 14.7: Text fields

Now, we have completed the styling of our frontend using MUI components.